

Misión Órbita: Avance 2 – Ejecución de la misión

Josue Monge Miranda

Escuela de Ingeniería del Software, Universidad Cenfotec

SOFT-01 Principios de programación 1, sección SCV3

Verónica Mora Lezcano

8 de noviembre de 2026

Repositorio: https://github.com/jmonge12/Misi-n-rbita---Avance-2/tree/main

Rama: main

Misión Órbita: Avance 2 – Ejecución de la misión

Este documento presenta el segundo avance del proyecto Misión Órbita, el simulador básico de exploración espacial hecho en Python. En el primer avance se prepararon los recursos de la misión; en esta etapa se trabaja lo que pasa cuando la nave ya llegó al destino: las actividades de exploración científica y la evaluación del regreso (Mora Lezcano, 2026).

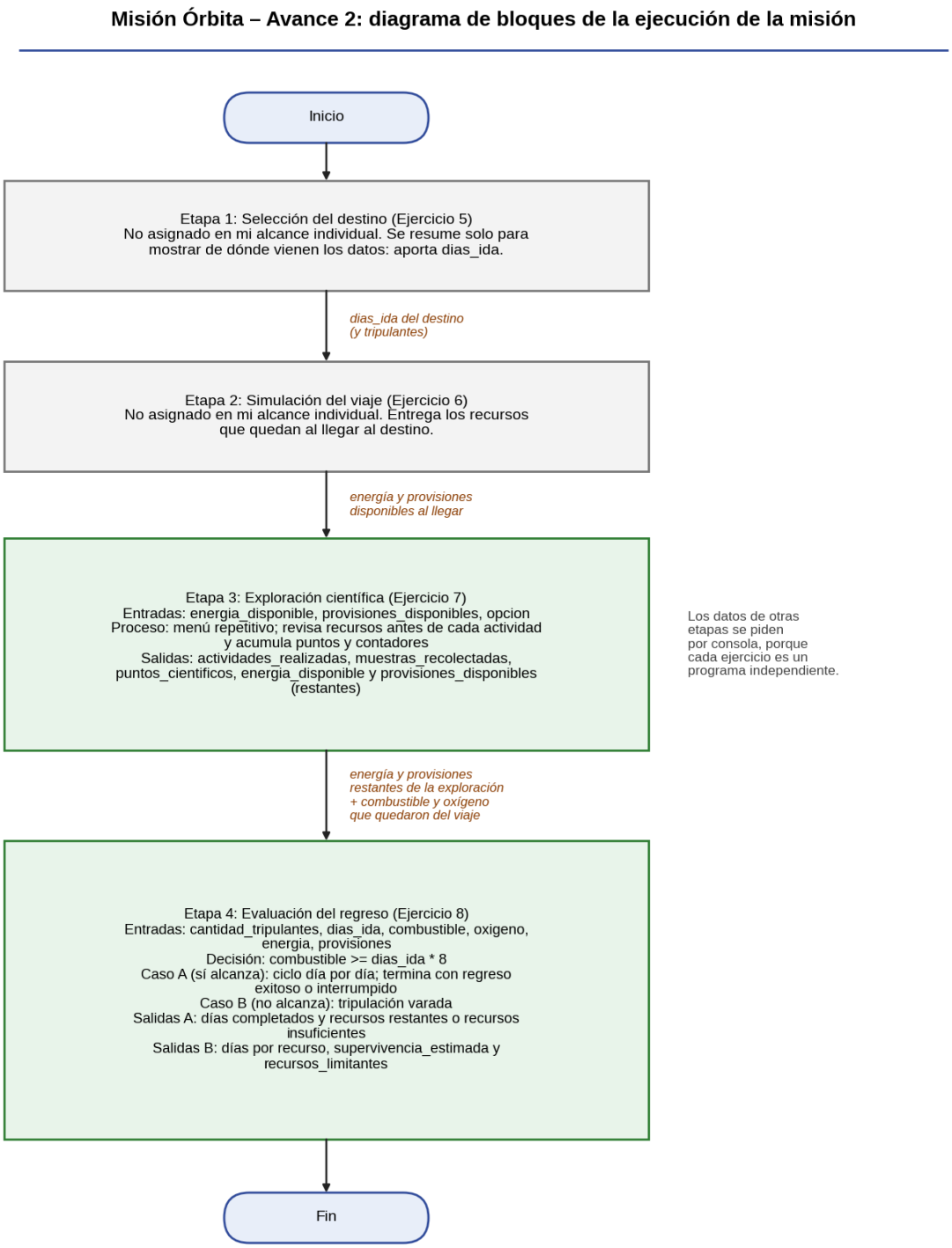
Como el trabajo es individual, el alcance corresponde a los ejercicios 7 y 8. Cada ejercicio se analizó antes de programarlo, así que se presentan su análisis de entrada, proceso y salida, su tabla de variables, su diagrama de flujo, el código fuente y los casos de prueba con la evidencia de ejecución. Todo respeta las restricciones técnicas de la consigna: solo consola, sin listas ni otras colecciones, sin funciones propias, sin importar módulos, sin funciones para obtener mínimos o máximos y sin instrucciones que corten los ciclos a la fuerza, de modo que cada ciclo termina por su condición.

Diagrama general del problema

La Figura 1 muestra cómo se conectan las cuatro etapas de la misión en este avance: destino, viaje, exploración y regreso. Las etapas 1 y 2 (ejercicios 5 y 6) no forman parte de mi alcance individual, por eso aparecen en gris y solo se indican para ver de dónde vienen los datos. Las etapas 3 y 4 (ejercicios 7 y 8) son las que desarrollo, con sus entradas y salidas, y las flechas indican los datos que pasan de una etapa a otra. Como cada ejercicio es un programa independiente, esos datos se piden por consola en lugar de recibirse de la etapa anterior.

Figura 1

Diagrama de bloques de la ejecución de la misión

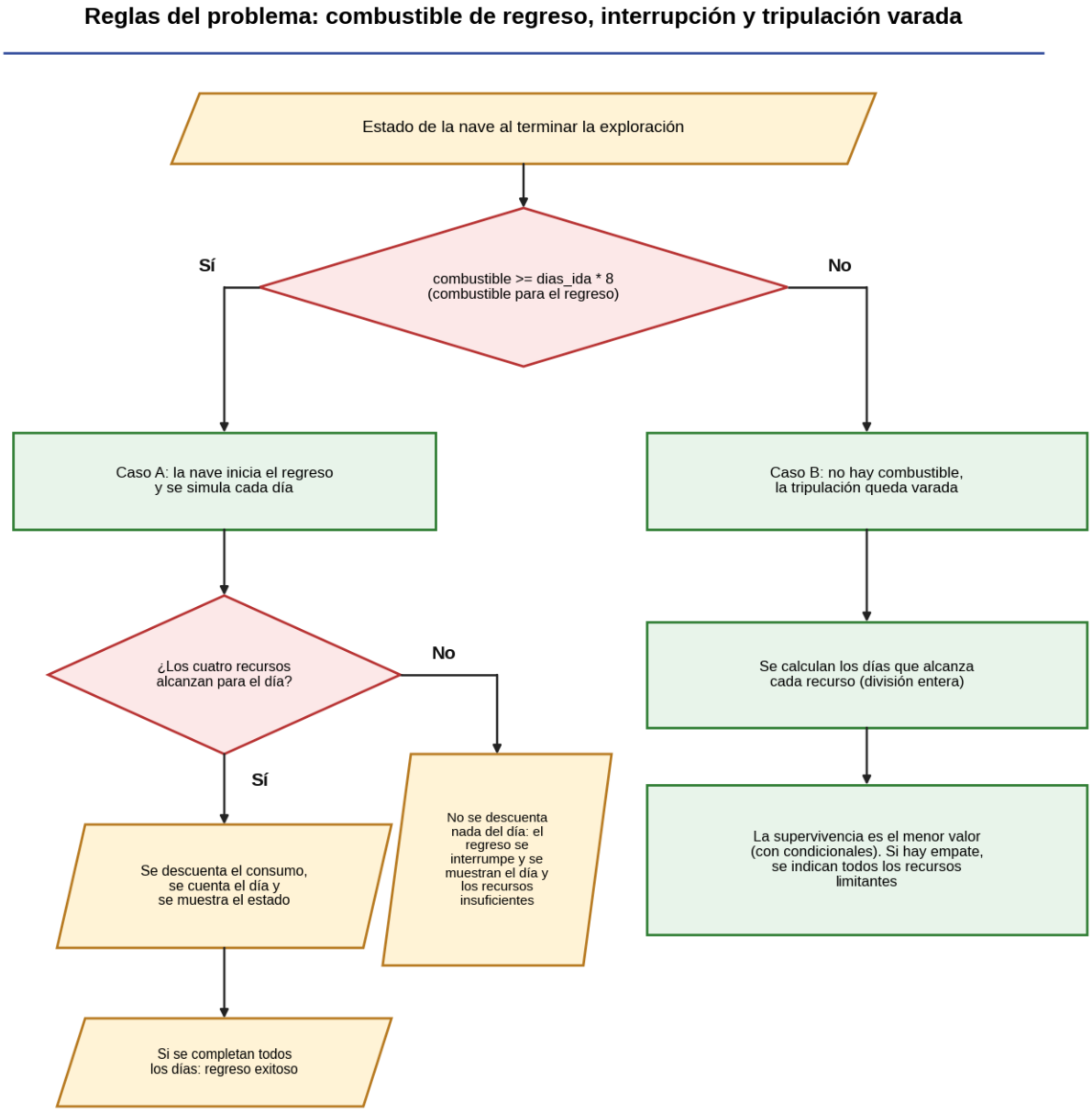


La Figura 2 resume las reglas del problema que los programas tienen que respetar. La primera es que no basta con tener combustible para llegar: para iniciar el regreso hace falta al

menos `dias_ida` * 8 unidades, y en ese cálculo no se suma la reserva de emergencia. Si alcanza, el regreso se simula día por día y se interrumpe en el primer día en que algún recurso no alcanza, sin descontar nada de ese día. Si no alcanza, la tripulación queda varada y se estima cuántos días puede sobrevivir con el oxígeno, la energía y las provisiones que quedan. La misma idea de revisar antes de consumir se aplica en la exploración, donde una actividad sin recursos suficientes termina el menú sin modificar el estado.

Figura 2

Reglas del problema: combustible de regreso, interrupción y tripulación varada



La misma regla de revisar antes de consumir se usa en la exploración (Ejercicio 7): si una actividad no tiene recursos suficientes, no se modifica nada y la exploración termina.

Diagrama explicativo

Las Tablas 1 y 2 presentan el análisis entrada, proceso y salida de cada ejercicio. Este análisis es la base para armar después las tablas de variables y los diagramas de flujo.

Tabla 1
Análisis entrada, proceso y salida del ejercicio 7

Entrada	Proceso	Salida
`energia_disponible` y `provisiones_disponibles` al llegar al destino. En cada vuelta del menú: `opcion`.	1. Se declaran las constantes y se inicializan `puntos_cientificos`, `actividades_realizadas` y `muestras_recolectadas` en 0 y `explorando` en True. 2. Mientras `explorando` sea True: se muestra el menú y se lee `opcion`. 3. Una condicional múltiple atiende la opción. En las opciones 1, 2 y 3 una condicional anidada revisa si alcanzan la energía y las provisiones. 4. Si alcanzan: se descuentan los recursos, se cuenta la actividad (y la muestra si es la opción 2) y se acumulan los puntos. 5. Si no alcanzan: se avisa, no se modifica nada y `explorando` pasa a False. 6. Opción 4: se avisa y `explorando` pasa a False. Cualquier otra opción: "Opción no válida." sin cambiar el estado.	En cada vuelta: mensaje de la actividad realizada con sus puntos, o el mensaje de rechazo, de finalización o de opción no válida. Al terminar: actividades realizadas, muestras recolectadas, puntos científicos, energía restante y provisiones restantes.

Tabla 2
Análisis entrada, proceso y salida del ejercicio 8

Entrada	Proceso	Salida
`cantidad_tripulantes`, `dias_ida`, `combustible`, `oxigeno`, `energia` y `provisiones`.	1. Se calculan `combustible_regreso` (`dias_ida` * 8), `oxigeno_diario` y `provisiones_diarias`. 2. Una condicional elige: si `combustible` >= `combustible_regreso` es el caso A; si no, el caso B. 3. Caso A: un ciclo repite cada día mientras `puede_continuar` sea True y `dias_completados` < `dias_ida`. Si los cuatro recursos alcanzan, se descuentan y se cuenta el día; si no, `puede_continuar` pasa a False. Al terminar el ciclo se compara `dias_completados` con `dias_ida`. 4. Caso B: se calculan los días por oxígeno, energía y provisiones con división entera, se obtiene el menor con condicionales y se arma el texto de los recursos limitantes.	Caso A: una línea por cada día completado; si se completan todos, mensaje de regreso exitoso y recursos restantes; si no, día interrumpido, recursos insuficientes y mensaje final. Caso B: mensaje de tripulación varada, días por cada recurso, supervivencia estimada y recurso(s) limitante(s).

Ejercicio 7: Actividades de exploración científica

Descripción de la solución

Este programa recibe la energía y las provisiones con las que la nave llega al destino y presenta un menú de actividades que se repite. Para lograrlo usa un ciclo `while` controlado por la variable booleana `explorando`, una condicional múltiple (`if / elif / else`) para atender la opción del menú, y contadores y acumuladores para llevar el estado de la exploración: `actividades_realizadas` y `muestras_recolectadas` son contadores y `puntos_cientificos` es un acumulador (Mora Lezcano, 2026).

Dentro de cada opción de actividad hay una condicional anidada que revisa si alcanzan la energía y las provisiones antes de descontar. La revisión va dentro de la rama porque solo tiene sentido después de saber qué actividad se eligió. Si no alcanzan, el programa avisa, deja todo sin modificar y cambia `explorando` a False, así que el menú termina aunque otra actividad más barata sí pudiera hacerse. La opción 4 también termina el menú, y cualquier otra opción muestra "Opción no válida." y repite el menú sin tocar el estado.

Las comparaciones usan >= porque tener justo lo necesario es suficiente, y eso se comprueba en el caso límite CP-07-03. Analizar terreno y realizar observación no gastan provisiones; aun así se compara con la constante correspondiente (valor 0) para que el código siga la tabla de la consigna y no queden números sueltos en las condiciones.

Tabla de variables

La Tabla 3 recoge todas las variables y constantes del programa con el mismo nombre que tienen en el código y en el diagrama. En la clasificación se indica entre paréntesis cuáles cumplen el papel de contador, acumulador o variable de control.

Tabla 3
Variables y constantes del ejercicio 7

Nombre	Tipo de dato	Clasificación	Descripción	Valor inicial / restricción
OPCION_ANALIZAR_TERRENO	int	Constante	Número de opción del menú para analizar terreno.	1
OPCION_RECOLECTAR_MUESTRA	int	Constante	Número de opción para recolectar muestra.	2
OPCION_REALIZAR_OBSERVACION	int	Constante	Número de opción para realizar observación.	3
OPCION_FINALIZAR	int	Constante	Número de opción para finalizar la exploración.	4
ENERGIA_ANALIZAR_TERRENO	int	Constante	Energía que gasta analizar terreno.	8
PROVISIONES_ANALIZAR_TERRENO	int	Constante	Provisiones que gasta analizar terreno.	0
PUNTOS_ANALIZAR_TERRENO	int	Constante	Puntos que da analizar terreno.	10
ENERGIA_RECOLECTAR_MUESTRA	int	Constante	Energía que gasta recolectar muestra.	12
PROVISIONES_RECOLECTAR_MUESTRA	int	Constante	Provisiones que gasta recolectar muestra.	2
PUNTOS_RECOLECTAR_MUESTRA	int	Constante	Puntos que da recolectar muestra.	20
ENERGIA_REALIZAR_OBSERVACION	int	Constante	Energía que gasta realizar observación.	5
PROVISIONES_REALIZAR_OBSERVACION	int	Constante	Provisiones que gasta realizar observación.	0
PUNTOS_REALIZAR_OBSERVACION	int	Constante	Puntos que da realizar observación.	8
energia_disponible	int	Entrada	Energía de la nave; se actualiza cada vez que se realiza una actividad.	Entero positivo
provisiones_disponibles	int	Entrada	Provisiones de la nave; se actualizan cuando se recolecta una muestra.	Entero positivo
opcion	int	Entrada	Opción del menú elegida en cada vuelta del ciclo.	Se valida contra 1, 2, 3 y 4
puntos_cientificos	int	Salida (acumulador)	Total de puntos obtenidos en la exploración.	0
actividades_realizadas	int	Salida (contador)	Cantidad de actividades que sí se realizaron.	0
muestras_recolectadas	int	Salida (contador)	Cantidad de muestras recolectadas; solo aumenta con la opción 2.	0
explorando	bool	Intermedia (variable de control)	Indica si el menú sigue repitiéndose; pasa a False al finalizar o al faltar recursos.	True

Diagrama de flujo

El diagrama se dividió en dos partes para que se lea bien. La Figura 3 muestra la inicialización, el ciclo con su condición y la cadena de decisiones del menú; la Figura 4 muestra la condicional anidada de cada actividad. Los conectores A, B y C llevan a cada actividad y el conector R devuelve el flujo a la condición del ciclo, que es donde se actualiza la variable de control.

Figura 3

Diagrama de flujo del ejercicio 7 (parte 1 de 2)

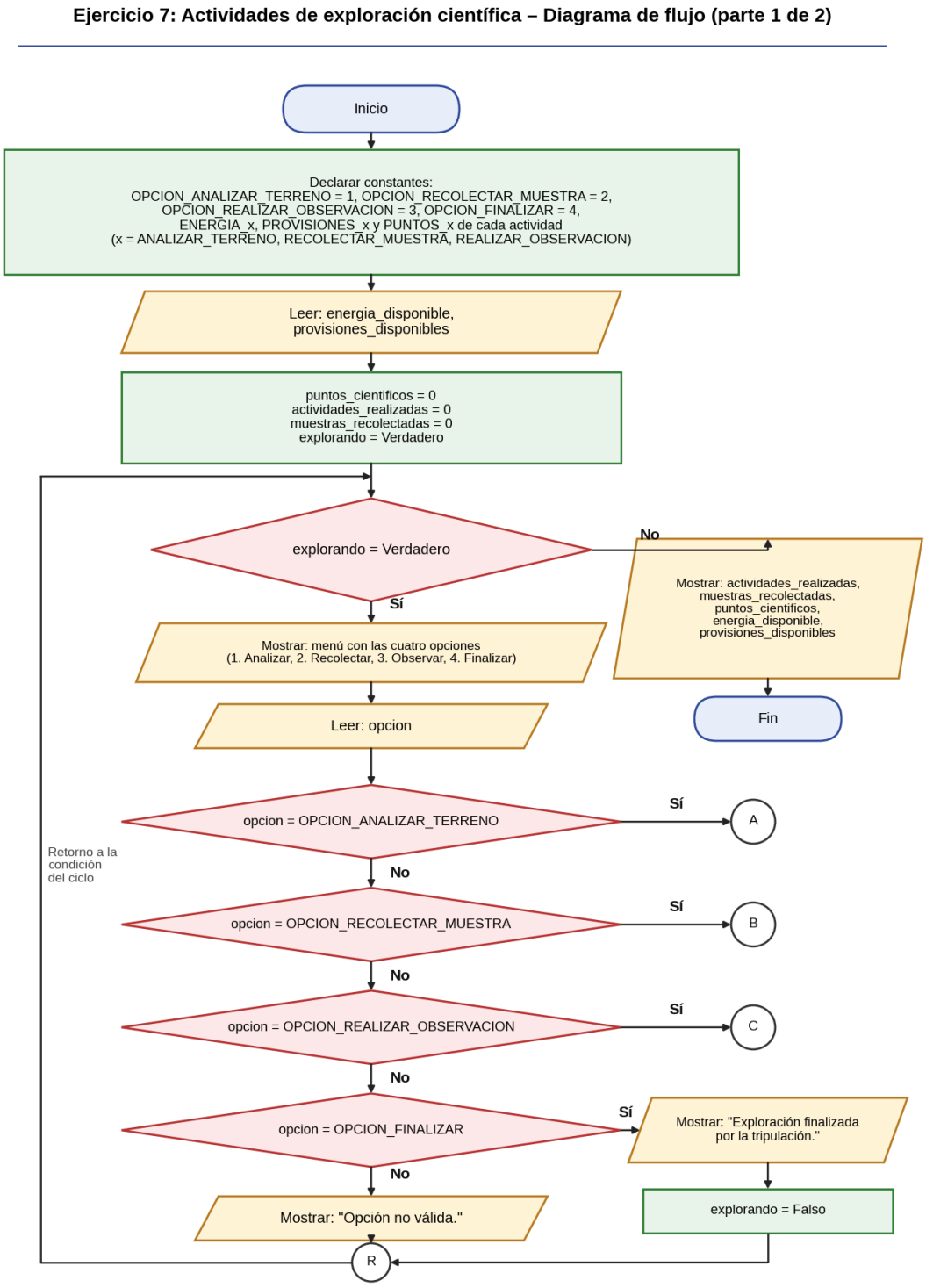
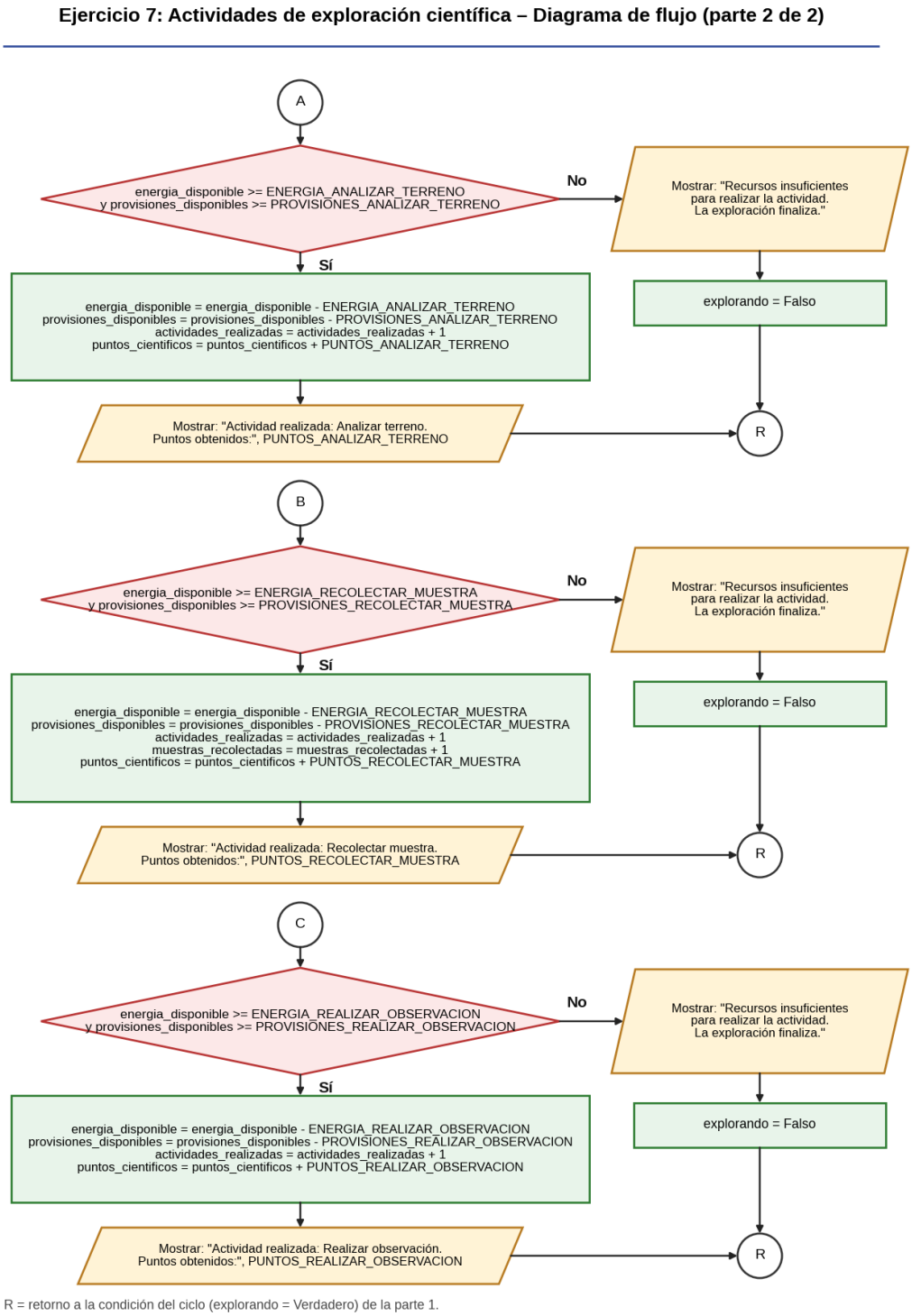


Figura 4

Diagrama de flujo del ejercicio 7 (parte 2 de 2)



R = retorno a la condición del ciclo (explorando = Verdadero) de la parte 1.

Código fuente

Figura 5

Código del archivo ejercicio7_exploracion.py

```
# ======================================================================
# Archivo: ejercicio7_exploracion.py
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV3
# Integrantes: Josue Monge Miranda
# Fecha: 08/11/2026    Versión: 1.0
# Descripción: Muestra un menú repetitivo de actividades de exploración
#              científica y acumula los puntos obtenidos.
# ======================================================================


# --- Declaración de constantes ---
# Las opciones y los costos van aquí para no dejar números mágicos
# dentro de las condiciones ni de las fórmulas.
OPCION_ANALIZAR_TERRENO = 1
OPCION_RECOLECTAR_MUESTRA = 2
OPCION_REALIZAR_OBSERVACION = 3
OPCION_FINALIZAR = 4


ENERGIA_ANALIZAR_TERRENO = 8
PROVISIONES_ANALIZAR_TERRENO = 0
PUNTOS_ANALIZAR_TERRENO = 10


ENERGIA_RECOLECTAR_MUESTRA = 12
PROVISIONES_RECOLECTAR_MUESTRA = 2
PUNTOS_RECOLECTAR_MUESTRA = 20


ENERGIA_REALIZAR_OBSERVACION = 5
PROVISIONES_REALIZAR_OBSERVACION = 0
PUNTOS_REALIZAR_OBSERVACION = 8


# --- Entrada de datos ---
energia_disponible = int(input("Energía disponible al llegar al destino: "))
provisiones_disponibles = int(input("Provisiones disponibles al llegar al destino: "))


# --- Valores iniciales del estado de la exploración ---
puntos_cientificos = 0
actividades_realizadas = 0
muestras_recolectadas = 0
explorando = True


# --- Ciclo del menú ---
# El ciclo se repite mientras explorando sea True. Solo cambia a False
# cuando la tripulación elige finalizar o cuando una actividad no tiene
# recursos suficientes, así que el ciclo siempre termina por su condición.
while explorando:
    print()
    print("===== Menú de exploración =====")
    print("1. Analizar terreno")
    print("2. Recolectar muestra")
    print("3. Realizar observación")
    print("4. Finalizar exploración")
    opcion = int(input("Seleccione una opción: "))


    # --- Decisión múltiple: una rama por cada opción del menú ---
    if opcion == OPCION_ANALIZAR_TERRENO:
        # Decisión anidada: solo tiene sentido revisar los recursos
        # después de saber qué actividad se eligió.
        if (energia_disponible >= ENERGIA_ANALIZAR_TERRENO
                and provisiones_disponibles >= PROVISIONES_ANALIZAR_TERRENO):
            energia_disponible = energia_disponible - ENERGIA_ANALIZAR_TERRENO
            provisiones_disponibles = provisiones_disponibles - PROVISIONES_ANALIZAR_TERRENO
            actividades_realizadas = actividades_realizadas + 1
            puntos_cientificos = puntos_cientificos + PUNTOS_ANALIZAR_TERRENO
            print("Actividad realizada: Analizar terreno. Puntos obtenidos:",
                  PUNTOS_ANALIZAR_TERRENO)
        else:
            print("Recursos insuficientes para realizar la actividad. La exploración finaliza.")
            explorando = False
```

```
    elif opcion == OPCION_RECOLECTAR_MUESTRA:
        if (energia_disponible >= ENERGIA_RECOLECTAR_MUESTRA
                and provisiones_disponibles >= PROVISIONES_RECOLECTAR_MUESTRA):
            energia_disponible = energia_disponible - ENERGIA_RECOLECTAR_MUESTRA
            provisiones_disponibles = provisiones_disponibles - PROVISIONES_RECOLECTAR_MUESTRA
            actividades_realizadas = actividades_realizadas + 1
            muestras_recolectadas = muestras_recolectadas + 1
            puntos_cientificos = puntos_cientificos + PUNTOS_RECOLECTAR_MUESTRA
            print("Actividad realizada: Recolectar muestra. Puntos obtenidos:",
                  PUNTOS_RECOLECTAR_MUESTRA)
        else:
            print("Recursos insuficientes para realizar la actividad. La exploración finaliza.")
            explorando = False
    elif opcion == OPCION_REALIZAR_OBSERVACION:
        if (energia_disponible >= ENERGIA_REALIZAR_OBSERVACION
                and provisiones_disponibles >= PROVISIONES_REALIZAR_OBSERVACION):
            energia_disponible = energia_disponible - ENERGIA_REALIZAR_OBSERVACION
            provisiones_disponibles = provisiones_disponibles - PROVISIONES_REALIZAR_OBSERVACION
            actividades_realizadas = actividades_realizadas + 1
            puntos_cientificos = puntos_cientificos + PUNTOS_REALIZAR_OBSERVACION
            print("Actividad realizada: Realizar observación. Puntos obtenidos:",
                  PUNTOS_REALIZAR_OBSERVACION)
        else:
            print("Recursos insuficientes para realizar la actividad. La exploración finaliza.")
            explorando = False
    elif opcion == OPCION_FINALIZAR:
        print("Exploración finalizada por la tripulación.")
        explorando = False
    else:
        # El estado no se modifica y el menú se vuelve a mostrar.
        print("Opción no válida.")

# --- Salida de resultados ---
print()
print("Actividades realizadas:", actividades_realizadas)
print("Muestras recolectadas:", muestras_recolectadas)
print("Puntos científicos:", puntos_cientificos)
print("Energía restante:", energia_disponible, "unidades")
print("Provisiones restantes:", provisiones_disponibles, "unidades")
```

Casos de prueba

Se diseñaron cinco casos que cubren los escenarios obligatorios: varias actividades con acumulación de puntos (CP-07-01 y CP-07-02), una opción no válida (CP-07-02 y CP-07-05), una actividad rechazada sin modificar el estado (CP-07-04, y también el final de CP-07-01) y la finalización con la opción 4 (CP-07-02, CP-07-03 y CP-07-05). Los resultados esperados se calcularon a mano y se compararon con la salida del programa, que se muestra en la Figura 6.

Tabla 4
Casos de prueba del ejercicio 7

ID	Ejercicio	Escenario	Datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-07-01	Ej. 7	Varias actividades con acumulación de puntos y una actividad rechazada por falta de energía	Energía = 20 Provisiones = 2 Opciones: 2, 1, 3	Recolectar muestra: energía 20 - 12 = 8, provisiones 2 - 2 = 0, +20 puntos. Analizar terreno: energía 8 - 8 = 0, +10 puntos. Observación pide 5 y quedan 0: "Recursos insuficientes para realizar la actividad. La exploración finaliza." Actividades 2, muestras 1, puntos 30, energía 0, provisiones 0	Igual al esperado: actividades 2, muestras 1, puntos 30, energía 0, provisiones 0, con el mensaje de recursos insuficientes	Aprobado
CP-07-02	Ej. 7	Acumulación de puntos, opción no válida y finalización con la opción 4	Energía = 30 Provisiones = 5 Opciones: 1, 3, 7, 3, 4	Analizar terreno: energía 22, 10 puntos. Observación: energía 17, 18 puntos. Opción 7: "Opción no válida." sin cambios. Observación: energía 12, 26 puntos. Opción 4: "Exploración finalizada por la tripulación." Actividades 3, muestras 0, puntos 26, energía 12, provisiones 5	Igual al esperado: actividades 3, muestras 0, puntos 26, energía 12, provisiones 5	Aprobado
CP-07-03	Ej. 7	Caso límite: recursos exactamente iguales al costo de la actividad	Energía = 12 Provisiones = 2 Opciones: 2, 4	Recolectar muestra: 12 >= 12 y 2 >= 2, se realiza. Energía 0, provisiones 0, +20 puntos. Opción 4: finaliza. Actividades 1, muestras 1, puntos 20, energía 0, provisiones 0	Igual al esperado: actividades 1, muestras 1, puntos 20, energía 0, provisiones 0	Aprobado
CP-07-04	Ej. 7	Actividad rechazada por provisiones, sin modificar el estado	Energía = 50 Provisiones = 1 Opción: 2	Recolectar muestra pide 2 provisiones y hay 1: "Recursos insuficientes para realizar la actividad. La exploración finaliza." El menú termina aunque analizar terreno sí podría hacerse. Actividades 0, muestras 0, puntos 0, energía 50, provisiones 1	Igual al esperado: actividades 0, muestras 0, puntos 0, energía 50, provisiones 1	Aprobado
CP-07-05	Ej. 7	Opción no válida al inicio y finalización sin actividades	Energía = 10 Provisiones = 3 Opciones: 0, 4	Opción 0: "Opción no válida." y el menú se repite. Opción 4: "Exploración finalizada por la tripulación." Actividades 0, muestras 0, puntos 0, energía 10, provisiones 3	Igual al esperado: actividades 0, muestras 0, puntos 0, energía 10, provisiones 3	Aprobado

Figura 6

Salida de consola de los casos CP-07-01 a CP-07-05

```
--- CP-07-01 ---
Energía disponible al llegar al destino: 20
Provisiones disponibles al llegar al destino: 2


===== Menú de exploración =====
1. Analizar terreno
2. Recolectar muestra
3. Realizar observación
4. Finalizar exploración
Seleccione una opción: 2
Actividad realizada: Recolectar muestra. Puntos obtenidos: 20


===== Menú de exploración =====
1. Analizar terreno
2. Recolectar muestra
3. Realizar observación
4. Finalizar exploración
Seleccione una opción: 1
Actividad realizada: Analizar terreno. Puntos obtenidos: 10


===== Menú de exploración =====
1. Analizar terreno
2. Recolectar muestra
3. Realizar observación
4. Finalizar exploración
Seleccione una opción: 3
Recursos insuficientes para realizar la actividad. La exploración finaliza.


Actividades realizadas: 2
Muestras recolectadas: 1
Puntos científicos: 30
Energía restante: 0 unidades
Provisiones restantes: 0 unidades


--- CP-07-02 ---
Energía disponible al llegar al destino: 30
Provisiones disponibles al llegar al destino: 5


===== Menú de exploración =====
1. Analizar terreno
2. Recolectar muestra
3. Realizar observación
4. Finalizar exploración
Seleccione una opción: 1
Actividad realizada: Analizar terreno. Puntos obtenidos: 10


===== Menú de exploración =====
1. Analizar terreno
2. Recolectar muestra
3. Realizar observación
4. Finalizar exploración
Seleccione una opción: 3
Actividad realizada: Realizar observación. Puntos obtenidos: 8


===== Menú de exploración =====
1. Analizar terreno
2. Recolectar muestra
3. Realizar observación
4. Finalizar exploración
Seleccione una opción: 7
Opción no válida.


===== Menú de exploración =====
1. Analizar terreno
2. Recolectar muestra
3. Realizar observación
```

```
4. Finalizar exploración
Seleccione una opción: 3
Actividad realizada: Realizar observación. Puntos obtenidos: 8

===== Menú de exploración =====
1. Analizar terreno
2. Recolectar muestra
3. Realizar observación
4. Finalizar exploración
Seleccione una opción: 4
Exploración finalizada por la tripulación.

Actividades realizadas: 3
Muestras recolectadas: 0
Puntos científicos: 26
Energía restante: 12 unidades
Provisiones restantes: 5 unidades

--- CP-07-03 ---
Energía disponible al llegar al destino: 12
Provisiones disponibles al llegar al destino: 2

===== Menú de exploración =====
1. Analizar terreno
2. Recolectar muestra
3. Realizar observación
4. Finalizar exploración
Seleccione una opción: 2
Actividad realizada: Recolectar muestra. Puntos obtenidos: 20

===== Menú de exploración =====
1. Analizar terreno
2. Recolectar muestra
3. Realizar observación
4. Finalizar exploración
Seleccione una opción: 4
Exploración finalizada por la tripulación.

Actividades realizadas: 1
Muestras recolectadas: 1
Puntos científicos: 20
Energía restante: 0 unidades
Provisiones restantes: 0 unidades

--- CP-07-04 ---
Energía disponible al llegar al destino: 50
Provisiones disponibles al llegar al destino: 1

===== Menú de exploración =====
1. Analizar terreno
2. Recolectar muestra
3. Realizar observación
4. Finalizar exploración
Seleccione una opción: 2
Recursos insuficientes para realizar la actividad. La exploración finaliza.

Actividades realizadas: 0
Muestras recolectadas: 0
Puntos científicos: 0
Energía restante: 50 unidades
Provisiones restantes: 1 unidades

--- CP-07-05 ---
Energía disponible al llegar al destino: 10
Provisiones disponibles al llegar al destino: 3

===== Menú de exploración =====
1. Analizar terreno
2. Recolectar muestra
```

```
3. Realizar observación
4. Finalizar exploración
Seleccione una opción: 0
Opción no válida.


===== Menú de exploración =====
1. Analizar terreno
2. Recolectar muestra
3. Realizar observación
4. Finalizar exploración
Seleccione una opción: 4
Exploración finalizada por la tripulación.


Actividades realizadas: 0
Muestras recolectadas: 0
Puntos científicos: 0
Energía restante: 10 unidades
Provisiones restantes: 3 unidades
```

Ejercicio 8: Evaluación del regreso y supervivencia de la tripulación

Descripción de la solución

Este programa recibe el estado de la nave al terminar la exploración y decide si puede iniciar el regreso. El combustible necesario es `dias_ida` * 8, sin sumar la reserva de emergencia de 10 unidades, porque así lo indica la consigna para este ejercicio (Mora Lezcano, 2026). Una condicional doble elige entre el caso A, si el combustible alcanza, y el caso B, si no alcanza.

En el caso A un ciclo `while` simula cada día del regreso. Su condición considera las dos cosas a la vez: que `puede_continuar` siga en True y que `dias_completados` sea menor que `dias_ida`. En cada vuelta se verifican los cuatro recursos antes de restar el consumo; si alguno no alcanza, no se descuenta nada de ese día y `puede_continuar` pasa a False. Cuando el ciclo termina, una condicional compara `dias_completados` con `dias_ida` para saber si el regreso se completó o se interrumpió. En la lista de recursos insuficientes no se revisa el combustible porque ya se comprobó que alcanza: cada día gasta 8 y se tenía al menos `dias_ida` * 8.

En el caso B la tripulación queda varada y se calculan los días que aguanta cada recurso con división entera. Como la consigna pide obtener el menor valor con condicionales, se parte de `dias_oxigeno` y se reemplaza el valor si `dias_energia` o `dias_provisiones` son menores. Para los recursos limitantes se va armando un texto: cada recurso que empata con el mínimo se agrega, y una condicional anidada decide si va solo o separado por coma, de modo que siempre salgan en el orden oxígeno, energía y provisiones. Como `CONSUMO_PROVISIONES_DIA` vale 1, `provisiones // provisiones_diarias` equivale a `provisiones // cantidad_tripulantes`, que es la fórmula de la consigna.

Tabla de variables

La Tabla 5 presenta las variables y constantes del ejercicio 8. Igual que en el ejercicio anterior, se indica entre paréntesis el contador y la variable de control del ciclo.

Tabla 5
Variables y constantes del ejercicio 8

Nombre	Tipo de dato	Clasificación	Descripción	Valor inicial / restricción
CONSUMO_COMBUSTIBLE_DIA	int	Constante	Combustible que se consume por día de regreso.	8
CONSUMO_OXIGENO_DIA	int	Constante	Oxígeno que consume cada tripulante por día.	2
CONSUMO_ENERGIA_DIA	int	Constante	Energía que consume la nave por día.	5
CONSUMO_PROVISIONES_DIA	int	Constante	Provisiones que consume cada tripulante por día.	1
cantidad_tripulantes	int	Entrada	Número de tripulantes de la nave.	Entero positivo
dias_ida	int	Entrada	Días que duró el viaje de ida; el regreso dura lo mismo.	Entero positivo
combustible	int	Entrada	Combustible disponible; en el caso A se actualiza cada día.	Entero positivo
oxigeno	int	Entrada	Oxígeno disponible; en el caso A se actualiza cada día.	Entero positivo
energia	int	Entrada	Energía disponible; en el caso A se actualiza cada día.	Entero positivo
provisiones	int	Entrada	Provisiones disponibles; en el caso A se actualizan cada día.	Entero positivo
combustible_regreso	int	Intermedia	Combustible necesario para el regreso (sin reserva).	Calculado
oxigeno_diario	int	Intermedia	Oxígeno que consume toda la tripulación por día.	Calculado
provisiones_diarias	int	Intermedia	Provisiones que consume toda la tripulación por día.	Calculado
dias_completados	int	Intermedia (contador)	Días de regreso que se lograron completar.	0
puede_continuar	bool	Intermedia (variable de control)	Indica si el regreso puede seguir; pasa a False cuando un recurso no alcanza.	True
dia_interrumpido	int	Intermedia	Número del día que no se pudo completar (días completados + 1).	Calculado solo si se interrumpe
dias_oxigeno	int	Intermedia	Días completos que alcanza el oxígeno (caso B).	Calculado
dias_energia	int	Intermedia	Días completos que alcanza la energía (caso B).	Calculado
dias_provisiones	int	Intermedia	Días completos que alcanzan las provisiones (caso B).	Calculado
supervivencia_estimada	int	Salida	Menor de los tres resultados: días que sobrevive la tripulación.	Calculado
recursos_limitantes	str	Salida	Nombre(s) del recurso o recursos con el valor mínimo, separados por coma.	"" (texto vacío)

Diagrama de flujo

Este diagrama es más largo, así que se dividió en seis partes (Figuras 7 a 12) unidas con conectores. La parte 1 lee los datos y toma la decisión principal; los conectores A y B llevan al caso A y al caso B. La parte 2 contiene el ciclo del regreso con su inicialización, su condición, el cuerpo, la actualización y la flecha de retorno; el conector D lleva a la parte 3, que muestra el resultado del caso A. Las partes 4 y 5 resuelven el caso B (conectores G y H) y la parte 6 termina en el único símbolo de Fin, al que también llega el conector F del caso A.

Figura 7

Diagrama de flujo del ejercicio 8 (parte 1 de 6)

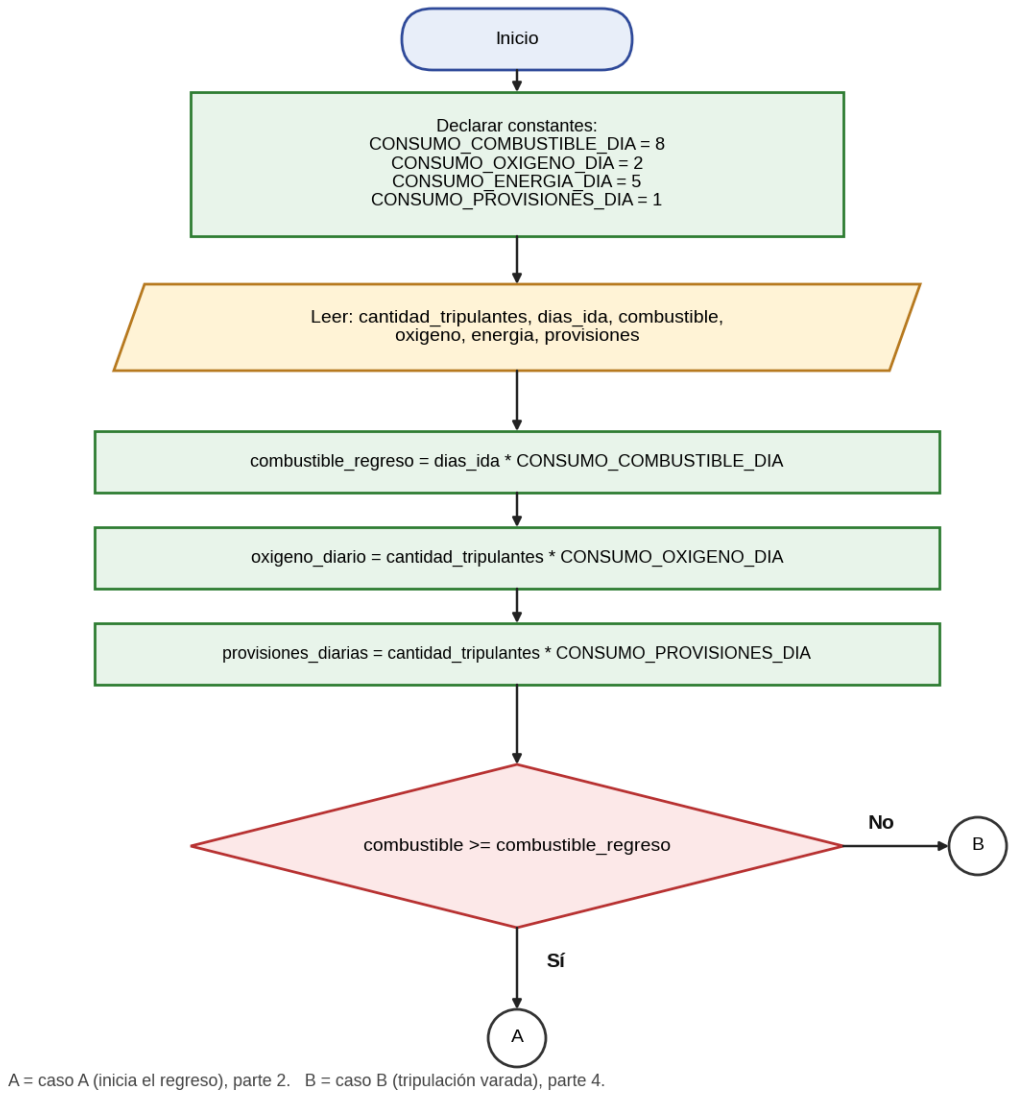
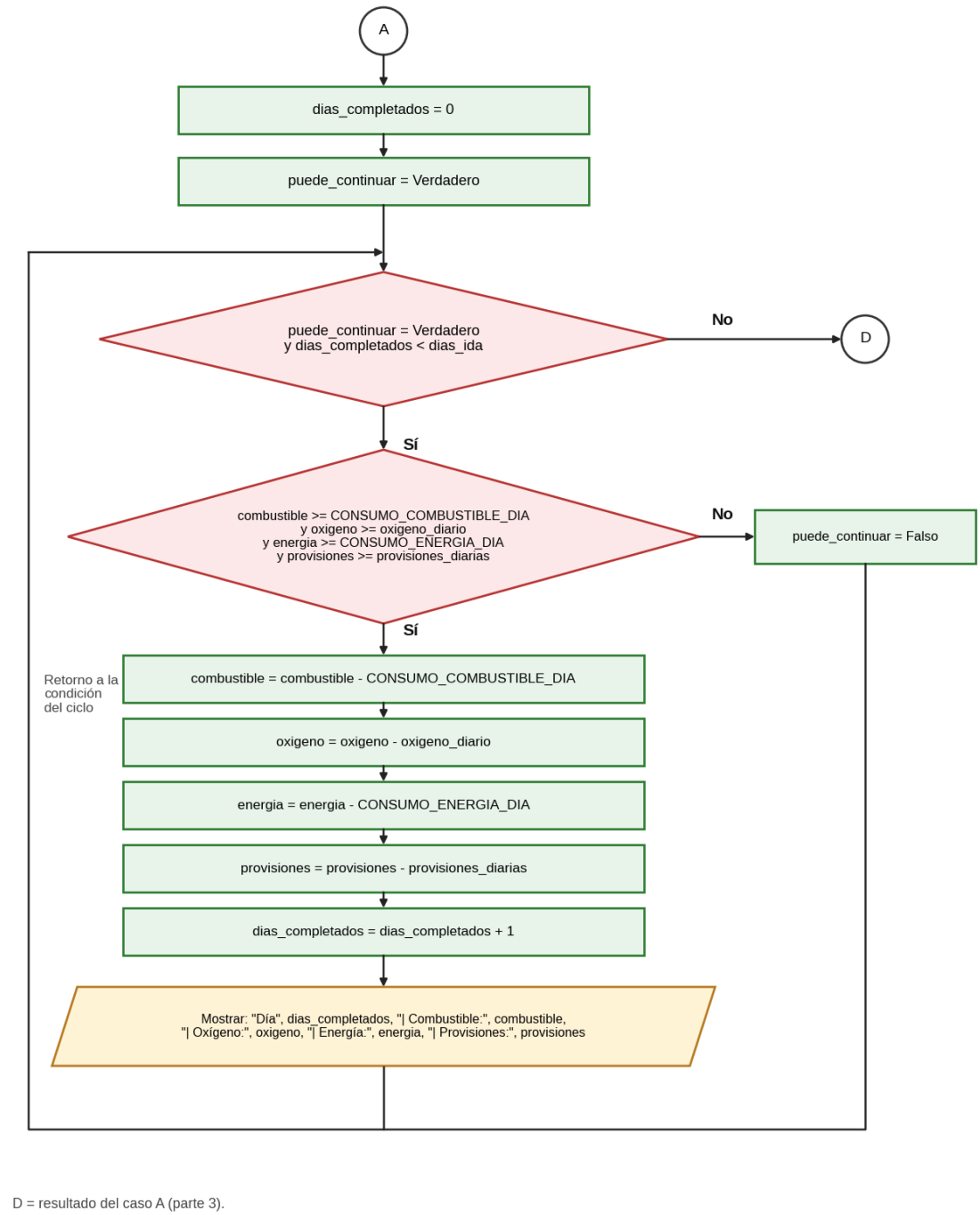


Figura 8

Diagrama de flujo del ejercicio 8 (parte 2 de 6)



D = resultado del caso A (parte 3).

Figura 9

Diagrama de flujo del ejercicio 8 (parte 3 de 6)



Figura 10

Diagrama de flujo del ejercicio 8 (parte 4 de 6)

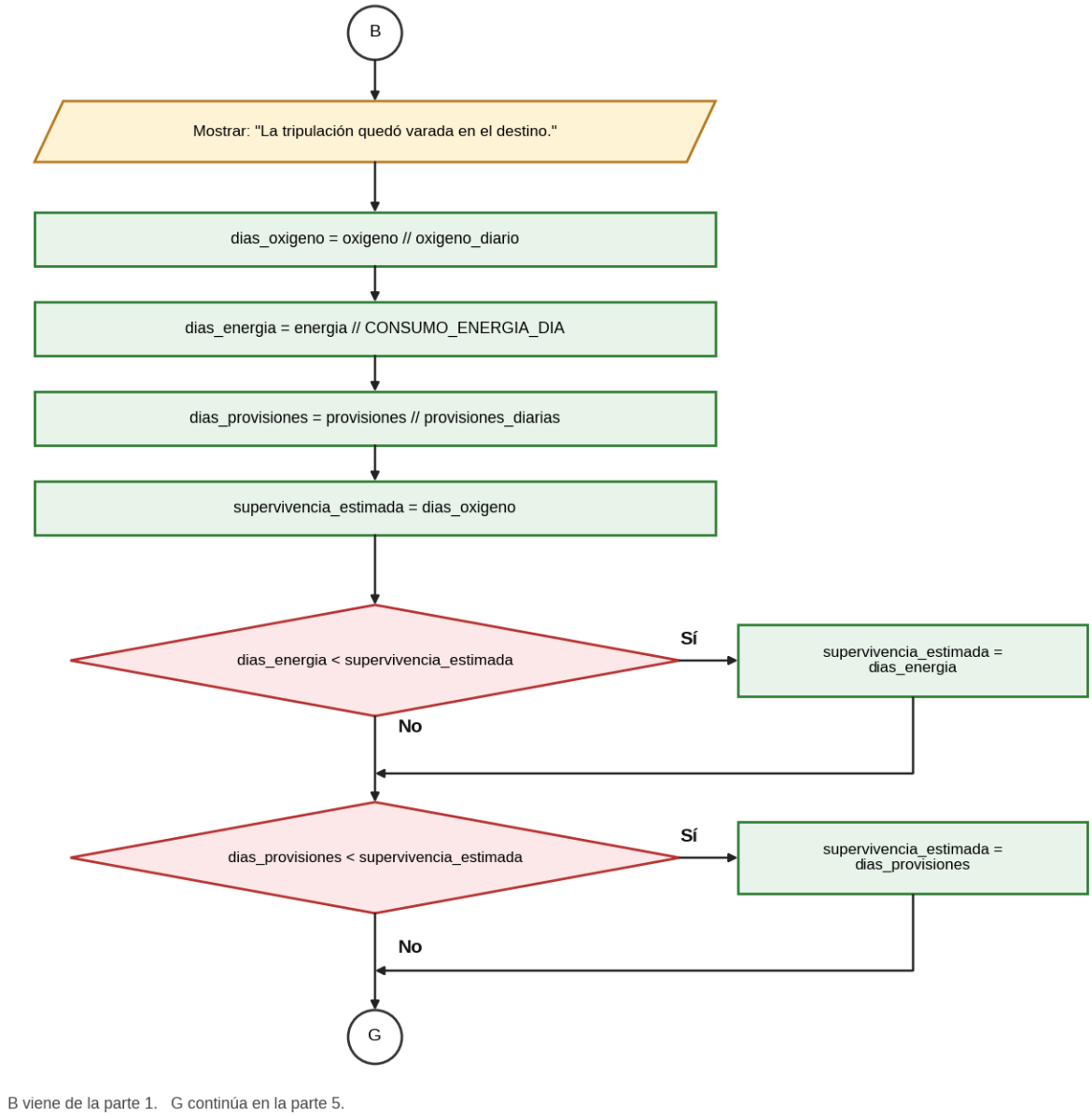


Figura 11

Diagrama de flujo del ejercicio 8 (parte 5 de 6)

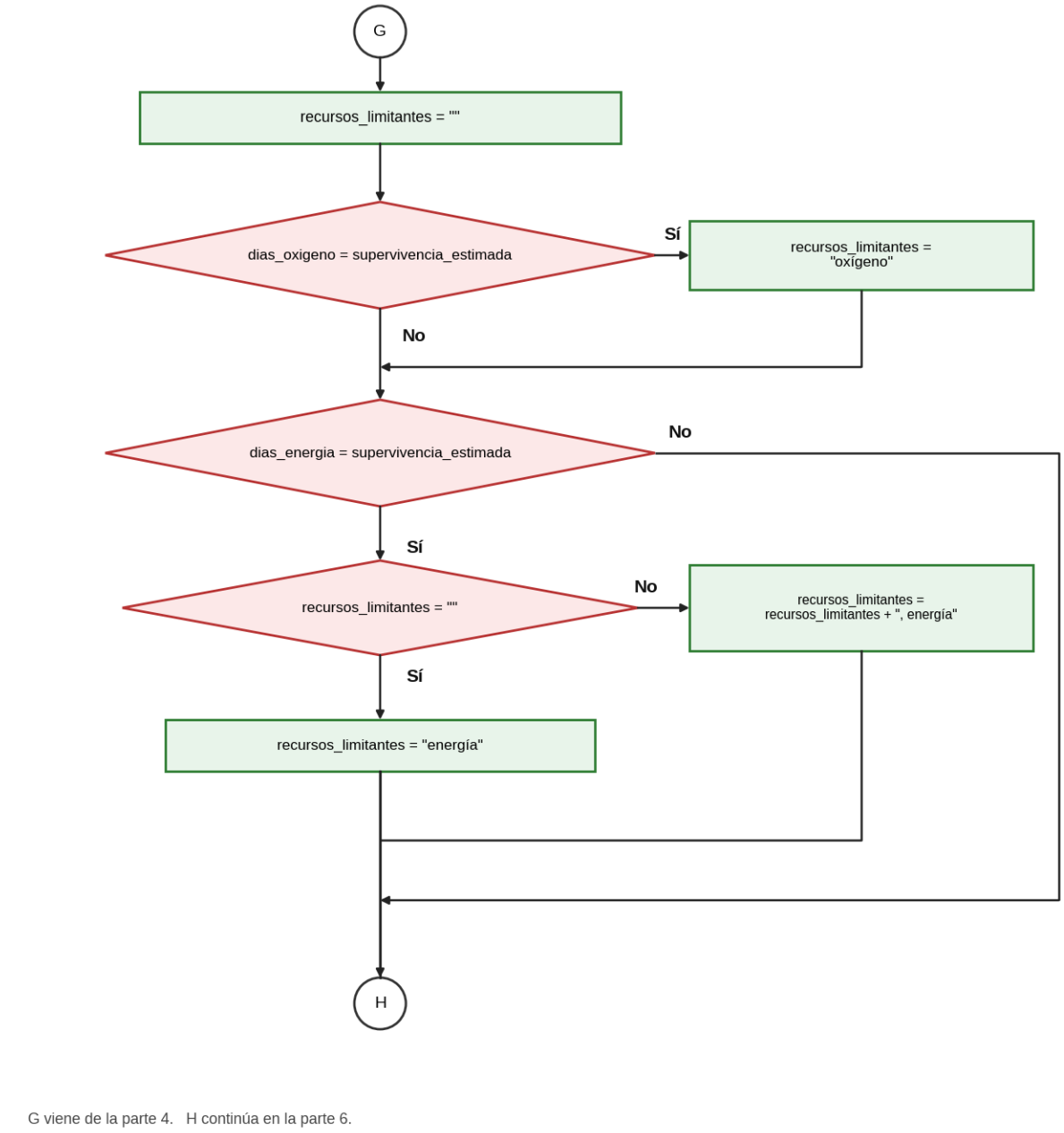
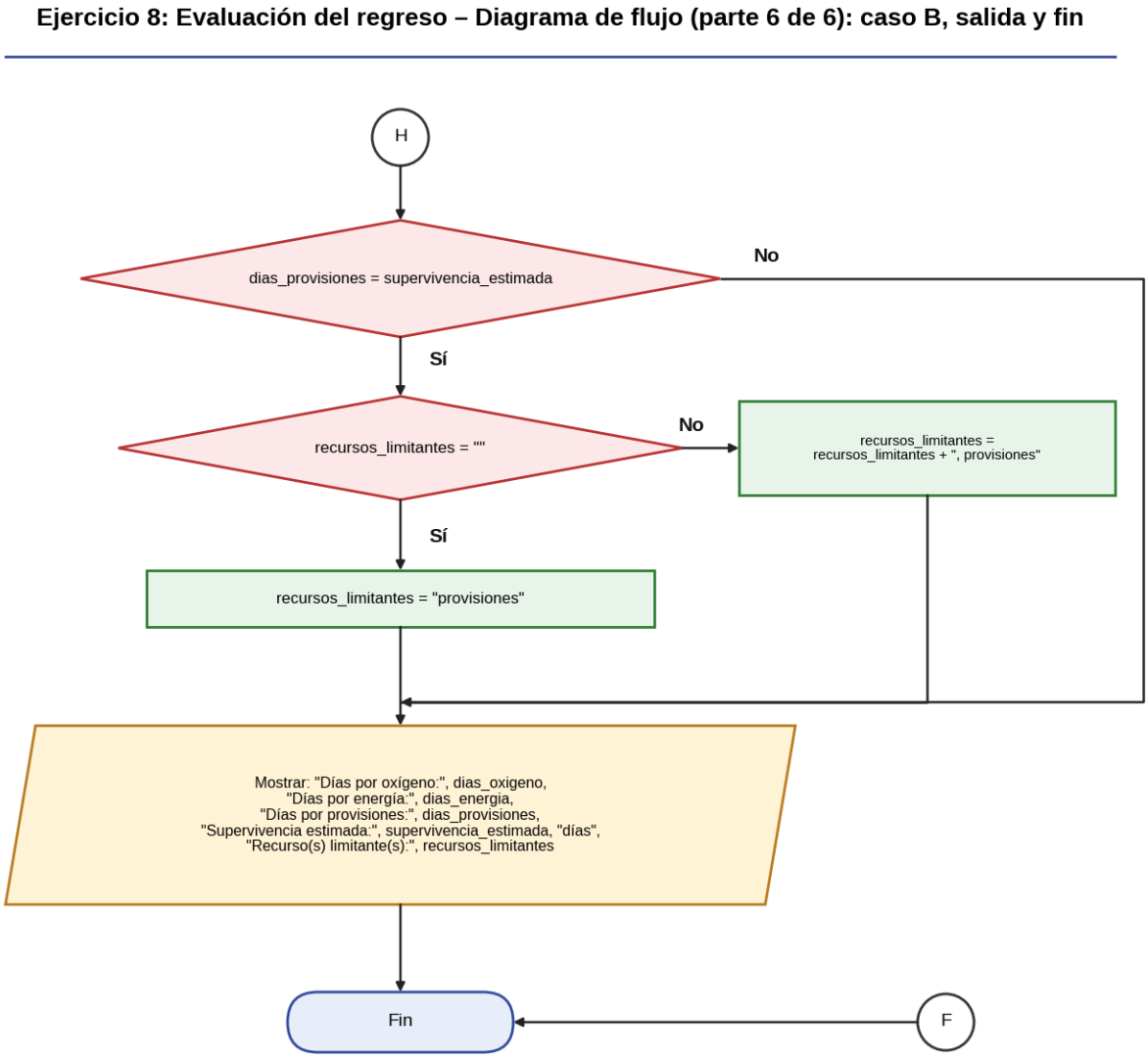


Figura 12
Diagrama de flujo del ejercicio 8 (parte 6 de 6)



H viene de la parte 5. F viene del caso A (parte 3).

Código fuente

Figura 13

Código del archivo ejercicio8_regreso.py

```
# ======================================================================
# Archivo: ejercicio8_regreso.py
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV3
# Integrantes: Josue Monge Miranda
# Fecha: 08/11/2026    Versión: 1.0
# Descripción: Evalúa si la nave puede iniciar el regreso. Si puede,
#              simula el viaje día por día; si no, calcula cuántos días
#              puede sobrevivir la tripulación varada.
# ======================================================================


# --- Declaración de constantes ---
CONSUMO_COMBUSTIBLE_DIA = 8
CONSUMO_OXIGENO_DIA = 2        # por tripulante
CONSUMO_ENERGIA_DIA = 5
CONSUMO_PROVISIONES_DIA = 1    # por tripulante


# --- Entrada de datos ---
cantidad_tripulantes = int(input("Cantidad de tripulantes: "))
dias_ida = int(input("Días del viaje de ida: "))
combustible = int(input("Combustible disponible: "))
oxigeno = int(input("Oxígeno disponible: "))
energia = int(input("Energía disponible: "))
provisiones = int(input("Provisiones disponibles: "))


# --- Procesamiento ---
# El regreso dura lo mismo que la ida. Aquí no se suma la reserva de
# emergencia, tal como lo indica la consigna.
combustible_regreso = dias_ida * CONSUMO_COMBUSTIBLE_DIA
oxigeno_diario = cantidad_tripulantes * CONSUMO_OXIGENO_DIA
provisiones_diarias = cantidad_tripulantes * CONSUMO_PROVISIONES_DIA


# --- Decisión: caso A (inicia el regreso) o caso B (tripulación varada) ---
if combustible >= combustible_regreso:
    # --- Caso A: simulación del regreso ---
    dias_completados = 0
    puede_continuar = True


    # El ciclo termina cuando se completan los días de la ida o cuando
    # un recurso no alcanza para el día siguiente.
    while puede_continuar and dias_completados < dias_ida:
        # Se verifica antes de descontar para no consumir nada de un día
        # que no se puede completar.
        if (combustible >= CONSUMO_COMBUSTIBLE_DIA
                and oxigeno >= oxigeno_diario
                and energia >= CONSUMO_ENERGIA_DIA
                and provisiones >= provisiones_diarias):
            combustible = combustible - CONSUMO_COMBUSTIBLE_DIA
            oxigeno = oxigeno - oxigeno_diario
            energia = energia - CONSUMO_ENERGIA_DIA
            provisiones = provisiones - provisiones_diarias
            dias_completados = dias_completados + 1
            print("Día", dias_completados, "| Combustible:", combustible,
                  "| Oxígeno:", oxigeno, "| Energía:", energia,
                  "| Provisiones:", provisiones)
        else:
            puede_continuar = False


    # --- Salida del caso A ---
    if dias_completados == dias_ida:
        print("La tripulación regresó exitosamente a la Tierra.")
        print("Combustible restante:", combustible, "unidades")
        print("Oxígeno restante:", oxigeno, "unidades")
        print("Energía restante:", energia, "unidades")
        print("Provisiones restantes:", provisiones, "unidades")
    else:
        # El día que falló es el siguiente al último día completado.
        dia_interrumpido = dias_completados + 1
        print("El regreso se interrumpió en el día " + str(dia_interrumpido) + ".")
```

```
        # El combustible no se revisa aquí porque ya se comprobó que alcanza.
        if oxigeno < oxigeno_diario:
            print("Recurso insuficiente: oxígeno")
        if energia < CONSUMO_ENERGIA_DIA:
            print("Recurso insuficiente: energía")
        if provisiones < provisiones_diarias:
            print("Recurso insuficiente: provisiones")
        print("La tripulación no completó el regreso.")
else:
    # --- Caso B: tripulación varada ---
    print("La tripulación quedó varada en el destino.")


    # División entera: solo cuentan los días completos.
    dias_oxigeno = oxigeno // oxigeno_diario
    dias_energia = energia // CONSUMO_ENERGIA_DIA
    dias_provisiones = provisiones // provisiones_diarias


    # El mínimo se obtiene con condicionales, como lo pide la consigna.
    supervivencia_estimada = dias_oxigeno
    if dias_energia < supervivencia_estimada:
        supervivencia_estimada = dias_energia
    if dias_provisiones < supervivencia_estimada:
        supervivencia_estimada = dias_provisiones


    # Si hay empate, se agregan todos los recursos con el valor mínimo,
    # siempre en el orden oxígeno, energía y provisiones.
    recursos_limitantes = ""
    if dias_oxigeno == supervivencia_estimada:
        recursos_limitantes = "oxígeno"
    if dias_energia == supervivencia_estimada:
        if recursos_limitantes == "":
            recursos_limitantes = "energía"
        else:
            recursos_limitantes = recursos_limitantes + ", energía"
    if dias_provisiones == supervivencia_estimada:
        if recursos_limitantes == "":
            recursos_limitantes = "provisiones"
        else:
            recursos_limitantes = recursos_limitantes + ", provisiones"


    # --- Salida del caso B ---
    print("Días por oxígeno:", dias_oxigeno)
    print("Días por energía:", dias_energia)
    print("Días por provisiones:", dias_provisiones)
    print("Supervivencia estimada:", supervivencia_estimada, "días")
    print("Recurso(s) limitante(s):", recursos_limitantes)
```

Casos de prueba

Los siete casos cubren los escenarios obligatorios: regreso exitoso (CP-08-01 y CP-08-04), regreso interrumpido por uno y por más de un recurso (CP-08-02 y CP-08-03) y tripulación varada con un solo recurso limitante (CP-08-05) y con empate (CP-08-06 y CP-08-07). Además, CP-08-04 prueba el límite exacto del combustible y CP-08-05 el caso en que falta una sola unidad. La salida real del programa está en la Figura 14.

Tabla 6
Casos de prueba del ejercicio 8

ID	Ejercicio	Escenario	Datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-08-01	Ej. 8	Regreso exitoso (caso A)	Tripulantes = 2 Días ida = 3 Combustible = 30 Oxígeno = 20 Energía = 20 Provisiones = 10	Necesario: 3 * 8 = 24 y 30 >= 24, caso A. Consumo diario: oxígeno 4, provisiones 2. Día 1: 22 | 16 | 15 | 8 Día 2: 14 | 12 | 10 | 6 Día 3: 6 | 8 | 5 | 4 "La tripulación regresó exitosamente a la Tierra." Restantes: 6, 8, 5 y 4	Igual al esperado: tres días completados y regreso exitoso con 6, 8, 5 y 4 unidades restantes	Aprobado
CP-08-02	Ej. 8	Regreso interrumpido por un solo recurso (oxígeno)	Tripulantes = 2 Días ida = 3 Combustible = 30 Oxígeno = 10 Energía = 20 Provisiones = 10	Caso A (30 >= 24). Día 1: 22 | 6 | 15 | 8 Día 2: 14 | 2 | 10 | 6 Día 3: oxígeno 2 < 4. "El regreso se interrumpió en el día 3." "Recurso insuficiente: oxígeno" "La tripulación no completó el regreso."	Igual al esperado: interrumpido en el día 3 con oxígeno como único recurso insuficiente	Aprobado
CP-08-03	Ej. 8	Regreso interrumpido por más de un recurso (oxígeno y energía)	Tripulantes = 2 Días ida = 3 Combustible = 30 Oxígeno = 10 Energía = 12 Provisiones = 10	Caso A (30 >= 24). Día 1: 22 | 6 | 7 | 8 Día 2: 14 | 2 | 2 | 6 Día 3: oxígeno 2 < 4 y energía 2 < 5; las provisiones (6 >= 2) alcanzan. "El regreso se interrumpió en el día 3." "Recurso insuficiente: oxígeno" "Recurso insuficiente: energía" y mensaje final	Igual al esperado: interrumpido en el día 3 con oxígeno y energía, en ese orden	Aprobado
CP-08-04	Ej. 8	Caso límite: combustible exactamente igual al necesario y recursos justos hasta quedar en cero	Tripulantes = 1 Días ida = 2 Combustible = 16 Oxígeno = 4 Energía = 10 Provisiones = 2	Necesario: 2 * 8 = 16 y 16 >= 16, caso A. Consumo diario: oxígeno 2, provisiones 1. Día 1: 8 | 2 | 5 | 1 Día 2: 0 | 0 | 0 | 0 Regreso exitoso con todos los recursos restantes en 0	Igual al esperado: dos días completados y los cuatro recursos en 0	Aprobado
CP-08-05	Ej. 8	Tripulación varada con un solo recurso limitante (falta una unidad de combustible)	Tripulantes = 2 Días ida = 5 Combustible = 39 Oxígeno = 20 Energía = 30 Provisiones = 12	Necesario: 5 * 8 = 40 y 39 < 40, caso B. Oxígeno: 20 // 4 = 5 Energía: 30 // 5 = 6 Provisiones: 12 // 2 = 6 Supervivencia estimada: 5 días. Limitante: oxígeno	Igual al esperado: 5, 6 y 6 días; supervivencia de 5 días con oxígeno como limitante	Aprobado
CP-08-06	Ej. 8	Tripulación varada con empate entre dos recursos	Tripulantes = 2 Días ida = 5 Combustible = 10 Oxígeno = 20 Energía = 25 Provisiones = 12	Necesario: 40 y 10 < 40, caso B. Oxígeno: 20 // 4 = 5 Energía: 25 // 5 = 5 Provisiones: 12 // 2 = 6 Supervivencia estimada: 5 días. Limitantes: oxígeno, energía	Igual al esperado: 5, 5 y 6 días; supervivencia de 5 días con oxígeno y energía como limitantes	Aprobado
CP-08-07	Ej. 8	Tripulación varada con empate entre los tres recursos	Tripulantes = 2 Días ida = 4 Combustible = 5 Oxígeno = 8 Energía = 10 Provisiones = 4	Necesario: 4 * 8 = 32 y 5 < 32, caso B. Oxígeno: 8 // 4 = 2 Energía: 10 // 5 = 2 Provisiones: 4 // 2 = 2 Supervivencia estimada: 2 días. Limitantes: oxígeno, energía, provisiones	Igual al esperado: 2, 2 y 2 días; supervivencia de 2 días con los tres recursos como limitantes	Aprobado

Figura 14

Salida de consola de los casos CP-08-01 a CP-08-07

```
--- CP-08-01 ---
Cantidad de tripulantes: 2
Días del viaje de ida: 3
Combustible disponible: 30
Oxígeno disponible: 20
Energía disponible: 20
Provisiones disponibles: 10
Día 1 | Combustible: 22 | Oxígeno: 16 | Energía: 15 | Provisiones: 8
Día 2 | Combustible: 14 | Oxígeno: 12 | Energía: 10 | Provisiones: 6
Día 3 | Combustible: 6 | Oxígeno: 8 | Energía: 5 | Provisiones: 4
La tripulación regresó exitosamente a la Tierra.
Combustible restante: 6 unidades
Oxígeno restante: 8 unidades
Energía restante: 5 unidades
Provisiones restantes: 4 unidades


--- CP-08-02 ---
Cantidad de tripulantes: 2
Días del viaje de ida: 3
Combustible disponible: 30
Oxígeno disponible: 10
Energía disponible: 20
Provisiones disponibles: 10
Día 1 | Combustible: 22 | Oxígeno: 6 | Energía: 15 | Provisiones: 8
Día 2 | Combustible: 14 | Oxígeno: 2 | Energía: 10 | Provisiones: 6
El regreso se interrumpió en el día 3.
Recurso insuficiente: oxígeno
La tripulación no completó el regreso.


--- CP-08-03 ---
Cantidad de tripulantes: 2
Días del viaje de ida: 3
Combustible disponible: 30
Oxígeno disponible: 10
Energía disponible: 12
Provisiones disponibles: 10
Día 1 | Combustible: 22 | Oxígeno: 6 | Energía: 7 | Provisiones: 8
Día 2 | Combustible: 14 | Oxígeno: 2 | Energía: 2 | Provisiones: 6
El regreso se interrumpió en el día 3.
Recurso insuficiente: oxígeno
Recurso insuficiente: energía
La tripulación no completó el regreso.


--- CP-08-04 ---
Cantidad de tripulantes: 1
Días del viaje de ida: 2
Combustible disponible: 16
Oxígeno disponible: 4
Energía disponible: 10
Provisiones disponibles: 2
Día 1 | Combustible: 8 | Oxígeno: 2 | Energía: 5 | Provisiones: 1
Día 2 | Combustible: 0 | Oxígeno: 0 | Energía: 0 | Provisiones: 0
La tripulación regresó exitosamente a la Tierra.
Combustible restante: 0 unidades
Oxígeno restante: 0 unidades
Energía restante: 0 unidades
Provisiones restantes: 0 unidades


--- CP-08-05 ---
Cantidad de tripulantes: 2
Días del viaje de ida: 5
Combustible disponible: 39
Oxígeno disponible: 20
Energía disponible: 30
Provisiones disponibles: 12
La tripulación quedó varada en el destino.
Días por oxígeno: 5
Días por energía: 6
Días por provisiones: 6
```

```
Supervivencia estimada: 5 días
Recurso(s) limitante(s): oxígeno


--- CP-08-06 ---
Cantidad de tripulantes: 2
Días del viaje de ida: 5
Combustible disponible: 10
Oxígeno disponible: 20
Energía disponible: 25
Provisiones disponibles: 12
La tripulación quedó varada en el destino.
Días por oxígeno: 5
Días por energía: 5
Días por provisiones: 6
Supervivencia estimada: 5 días
Recurso(s) limitante(s): oxígeno, energía


--- CP-08-07 ---
Cantidad de tripulantes: 2
Días del viaje de ida: 4
Combustible disponible: 5
Oxígeno disponible: 8
Energía disponible: 10
Provisiones disponibles: 4
La tripulación quedó varada en el destino.
Días por oxígeno: 2
Días por energía: 2
Días por provisiones: 2
Supervivencia estimada: 2 días
Recurso(s) limitante(s): oxígeno, energía, provisiones
```

Retroalimentación del docente

En esta entrega se corrigen las observaciones que la docente hizo al Avance 1. La Tabla 7 reúne cada observación, la corrección aplicada y el commit del repositorio en el que se hizo el cambio.

Tabla 7
Cambios realizados a partir de la retroalimentación del Avance 1

Observación del docente	Corrección aplicada	Commit
[completar con la observación 1]	[completar]	[completar]
[completar con la observación 2]	[completar]	[completar]
[completar con la observación 3]	[completar]	[completar]

Repositorio Git

El código y la documentación están en el repositorio de GitHub: https://github.com/jmonge12/Misi-n-rbita---Avance-1. Sigue la estructura sugerida en la consigna: la carpeta `avance1` con los archivos del primer avance, la carpeta `avance2` con `ejercicio7_exploracion.py` y `ejercicio8_regreso.py`, y la carpeta `documentacion` con los PDF. El desarrollo del avance se registra con commits de mensajes descriptivos. En la Figura 15 se muestra el contenido del README.md.

Figura 15
Contenido del archivo README.md del repositorio

```
# Misión Órbita


## Descripción
Simulador básico de exploración espacial desarrollado en Python para el curso
Principios de Programación 1 (SOFT-01, sección SCV3) de la Universidad CENFOTEC.
El proyecto se construye en tres avances:


- **Avance 1 – Preparación de la misión:** cuatro algoritmos independientes
  (recursos necesarios, combustible, soporte de la tripulación y autorización
  de lanzamiento).
- **Avance 2 – Ejecución de la misión:** actividades de exploración científica
  (ejercicio 7) y evaluación del regreso y supervivencia de la tripulación
  (ejercicio 8).


## Integrantes
- Josue Monge Miranda


## Estructura del repositorio
```
```
```

```
mision-orbita/
├── README.md
├── avance1/
│   ├── ejercicio1_recursos.py
│   ├── ejercicio2_combustible.py
│   ├── ejercicio3_soporte_tripulacion.py
│   └── ejercicio4_autorizacion.py
├── avance2/
│   ├── ejercicio7_exploracion.py
│   └── ejercicio8_regreso.py
└── documentacion/
    ├── avance1.pdf
    ├── avance2.pdf
    ├── ejercicio7_exploracion.pdf
    └── ejercicio8_regreso.pdf
```

Instrucciones de ejecución
Cada programa es independiente y se ejecuta desde la consola con Python 3:

```
python avance1/ejercicio1_recursos.py
python avance1/ejercicio2_combustible.py
python avance1/ejercicio3_soporte_tripulacion.py
python avance1/ejercicio4_autorizacion.py
python avance2/ejercicio7_exploracion.py
python avance2/ejercicio8_regreso.py
```

Todos los datos se ingresan por consola como números enteros positivos.
En el ejercicio 7 el menú se repite hasta elegir la opción 4 o hasta que una
actividad no pueda realizarse por falta de recursos.

Convenciones y estilos
- Variables en snake_case, con nombres descriptivos y en minúscula.
- Constantes en MAYÚSCULAS con guion bajo, declaradas al inicio del archivo.
- Sin números mágicos dentro de las fórmulas ni de las condiciones.
- Indentación de 4 espacios y espacios alrededor de los operadores.
- Encabezado en cada archivo (archivo, curso, integrantes, fecha, versión y descripción).
- Un comentario antes de cada bloque lógico: constantes, entrada, procesamiento,
  decisión y salida; los comentarios explican el porqué de las decisiones.
- Mensajes de consola con sus unidades.
- Commits con mensajes que describen el cambio realizado.

Referencias

Monge Miranda, J. (2026). *Misión Órbita: Avance 1 – Preparación de la misión* [Trabajo de curso no publicado]. Escuela de Ingeniería del Software, Universidad Cenfotec.

Mora Lezcano, V. (2026). *Consigna del segundo avance del proyecto: Ejecución de la misión* [Documento de curso]. Escuela de Ingeniería del Software, Universidad Cenfotec.